

Computing Convolution via the DFT, and Bluestein's Algorithm

Contents

1	Scope	1
2	How the DFT Computes Convolution	1
2.1	Setup	1
2.2	Full derivation	1
2.3	Numeric verification, $N = 4$	2
2.4	Why linear convolution needs zero-padding	3
3	Bluestein's Algorithm	3
3.1	The obstruction	3
3.2	The identity	3
3.3	Rewriting the exponent	3
3.4	Assembling the convolution	4
3.5	Why this must be a <i>linear</i> , not circular, convolution	4
3.6	Constructing the padded arrays	4
3.7	Full algorithm	5
3.8	Worked numeric example, $N = 3$ (not a power of two)	5
3.9	Why the inverse needs no separate implementation	5
4	Summary Table	6

1 Scope

This assumes you already know what convolution is. The focus here is on two mechanisms only:

1. **Exactly how** the DFT computes a convolution – the full derivation, including the one subtlety (index arithmetic mod N inside a complex exponential) that most treatments skip over.
2. **Exactly how** Bluestein's algorithm is built, step by step from the algebraic identity to the padded arrays you actually construct in code, verified against a hand-computable numeric example.

2 How the DFT Computes Convolution

2.1 Setup

Let $x[n]$ and $h[n]$ both have length N (zero-pad the shorter one first). Define the **circular** convolution:

$$y[m] = \sum_{n=0}^{N-1} x[n] h[(m - n) \bmod N], \quad m = 0, \dots, N - 1$$

Claim: if X, H, Y are the length- N DFTs of x, h, y , then

$$Y[k] = X[k] \cdot H[k] \quad \text{for every } k.$$

2.2 Full derivation

Take the DFT of y directly and substitute its definition:

$$Y[k] = \sum_{m=0}^{N-1} y[m] e^{-j2\pi km/N} = \sum_{m=0}^{N-1} \sum_{n=0}^{N-1} x[n] h[(m-n) \bmod N] e^{-j2\pi km/N}$$

Swap the order of summation (always legal for finite sums):

$$Y[k] = \sum_{n=0}^{N-1} x[n] \sum_{m=0}^{N-1} h[(m-n) \bmod N] e^{-j2\pi km/N}$$

Substitute $p = (m-n) \bmod N$ in the inner sum. As m ranges over $0, \dots, N-1$, p also ranges over the complete set $0, \dots, N-1$ (just in a cyclically shifted order), so the inner sum is still a full sum over p :

$$\sum_{m=0}^{N-1} h[(m-n) \bmod N] e^{-j2\pi km/N} = \sum_{p=0}^{N-1} h[p] e^{-j2\pi k(p+n)/N}$$

The subtlety that makes this legal. We wrote $m = p + n$, but really only $m \equiv p + n \pmod{N}$ – they can differ by a multiple of N . This is fine *only* because $e^{-j2\pi km/N}$ is periodic in m with period exactly N : adding N to m multiplies the exponent by $e^{-j2\pi k} = 1$. So the exponential cannot tell the difference between m and $m \bmod N$, and the substitution $m = p + n$ is valid even though it's only true modulo N . This periodicity is precisely why the DFT is a *circular*-arithmetic object, and it's the exact fact Bluestein's algorithm has to work around later (Section 3.4) when it needs a *linear*, non-wrapping convolution instead.

Continue by splitting the exponential $e^{-j2\pi k(p+n)/N} = e^{-j2\pi kp/N} \cdot e^{-j2\pi kn/N}$:

$$Y[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N} \sum_{p=0}^{N-1} h[p] e^{-j2\pi kp/N}$$

The two sums are now completely independent of each other – the first depends only on n , the second only on p – so they factor:

$$Y[k] = \left(\sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N} \right) \left(\sum_{p=0}^{N-1} h[p] e^{-j2\pi kp/N} \right) = X[k] \cdot H[k]$$

That factorization is the entire theorem. It only worked because (a) the exponential's periodicity let us treat m and $p + n$ as interchangeable, and (b) $e^{a+b} = e^a e^b$ let the sum split into a product of two independent sums.

2.3 Numeric verification, $N = 4$

Let $x = [1, 2, 3, 4]$, $h = [1, 0, 1, 0]$.

Direct circular convolution.

$$\begin{aligned}
y[0] &= x[0]h[0] + x[1]h[3] + x[2]h[2] + x[3]h[1] = 1 + 0 + 3 + 0 = 4 \\
y[1] &= x[0]h[1] + x[1]h[0] + x[2]h[3] + x[3]h[2] = 0 + 2 + 0 + 4 = 6 \\
y[2] &= x[0]h[2] + x[1]h[1] + x[2]h[0] + x[3]h[3] = 1 + 0 + 3 + 0 = 4 \\
y[3] &= x[0]h[3] + x[1]h[2] + x[2]h[1] + x[3]h[0] = 0 + 2 + 0 + 4 = 6
\end{aligned}$$

So $y = [4, 6, 4, 6]$.

Via DFT. $X = [10, -2 + 2j, -2, -2 - 2j]$ (the same $N = 4$ hand-computed DFT from earlier). H , the DFT of $[1, 0, 1, 0]$, is $[2, 0, 2, 0]$ – only the even bins survive, which makes sense since h itself has period 2. Multiply pointwise:

$$Y = X \cdot H = [10 \cdot 2, (-2 + 2j) \cdot 0, -2 \cdot 2, (-2 - 2j) \cdot 0] = [20, 0, -4, 0]$$

Inverse-DFT Y (sum and divide by $N = 4$): this returns exactly $[4, 6, 4, 6]$ – matching the direct computation. The theorem holds, verified arithmetically rather than just asserted.

2.4 Why linear convolution needs zero-padding

If you actually wanted the *linear* convolution of two length- N sequences (no wraparound – the version used for filtering or polynomial multiplication), computing it via DFT directly on the unpadded, length- N sequences gives the *circular* result instead, which corrupts the true output: the tail of the true linear convolution wraps around and adds into the front.

Fix: zero-pad both sequences to length $L \geq N_x + N_h - 1$ (the true linear-convolution output length) before taking DFTs. With enough trailing zeros, the wraparound only ever lands on zero-valued positions, so the circular result and the linear result become identical. This is not an approximation – it is an exact equivalence once the padding is long enough.

3 Bluestein's Algorithm

3.1 The obstruction

The ordinary DFT sum has a *product* of indices, kn , inside the exponent:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}$$

The convolution theorem above needed the exponent to depend on a *difference* of indices ($p + n$, which splits into independent factors) – a product kn does not split that way, since $e^{-j2\pi kn/N}$ cannot be written as (something depending only on k) times (something depending only on n) times (something depending only on a fixed offset). Bluestein's entire contribution is a purely algebraic rewrite that converts the product kn into a difference, so the same convolution machinery becomes applicable to *any* N , not just powers of two.

3.2 The identity

$$kn = \frac{n^2 + k^2 - (k - n)^2}{2}$$

Verify by expanding $(k - n)^2 = k^2 - 2kn + n^2$, so $n^2 + k^2 - (k - n)^2 = 2kn$; divide by 2.

3.3 Rewriting the exponent

Substitute into $-2\pi kn/N = -\pi(n^2 + k^2 - (k-n)^2)/N$ and split using $e^{a+b+c} = e^a e^b e^c$:

$$e^{-j2\pi kn/N} = \underbrace{e^{-j\pi n^2/N}}_{\text{depends on } n} \cdot \underbrace{e^{-j\pi k^2/N}}_{\text{depends on } k} \cdot \underbrace{e^{+j\pi(k-n)^2/N}}_{\text{depends on } (k-n)}$$

Define the **chirp** $c[n] = e^{-j\pi n^2/N}$ (its instantaneous phase grows quadratically – like an audio chirp sweeping frequency). Then:

$$e^{-j\pi n^2/N} = c[n], \quad e^{-j\pi k^2/N} = c[k], \quad e^{+j\pi(k-n)^2/N} = \overline{c[k-n]}$$

(the last equality holds because c has unit modulus, so flipping the sign of its exponent is the same as taking its complex conjugate).

3.4 Assembling the convolution

Substitute back into the DFT sum:

$$X[k] = \sum_{n=0}^{N-1} x[n] c[n] c[k] \overline{c[k-n]} = c[k] \sum_{n=0}^{N-1} (x[n] c[n]) \overline{c[k-n]}$$

Define $a[n] = x[n] c[n]$ and $b[m] = \overline{c[m]}$. Then:

$$X[k] = c[k] \cdot \sum_{n=0}^{N-1} a[n] b[k-n] = c[k] \cdot (a * b)[k]$$

This sum is exactly a convolution of a with b evaluated at k – the ”difference of indices” structure the previous section’s convolution machinery needs.

This is why the obstruction is gone. $c[k]$ and $c[n]$ separated cleanly from the mixed term because the identity in Section 3.2 rewrote the *product* kn as a sum of three independent *squares*. Each square depends on only one of $\{n, k, k-n\}$, which is exactly the ”depends on one variable at a time” structure that lets a sum factor – the same structural requirement the plain Convolution Theorem relied on in Section 2.2.

3.5 Why this must be a *linear*, not circular, convolution

In $\sum_n a[n] b[k-n]$, as k ranges $0, \dots, N-1$ and n ranges $0, \dots, N-1$, the argument $(k-n)$ ranges over $-(N-1)$ to $N-1$ – that’s $2N-1$ distinct values, not N . Recall from Section 2.4: computing this as an ordinary length- N *circular* convolution would let those negative-index values wrap around mod N and collide with the positive-index ones, corrupting the result. So this must be treated as a genuine linear convolution, computed via DFT/FFT only after zero-padding both a and b to length $M \geq 2N-1$ (rounded up to the next power of two so a radix-2 FFT can run on it).

3.6 Constructing the padded arrays

This is the part that trips people up in code, so make it fully explicit. Let $M = \text{next_power_of_two}(2N - 1)$.

Array a (length M): straightforward.

$$a[n] = x[n] c[n] \text{ for } n = 0, \dots, N - 1, \quad a[n] = 0 \text{ for } n = N, \dots, M - 1$$

Array b (length M): needs values of $\overline{c[m]}$ at $m = -(N - 1), \dots, N - 1$. Using the standard circular-array convention (index $-p$ is stored at array position $M - p$):

$$b[0] = \overline{c[0]} = 1$$

$$b[1], \dots, b[N - 1] = \overline{c[1]}, \dots, \overline{c[N - 1]} \quad (\text{positive-index values, stored at the front})$$

$$b[M - 1], b[M - 2], \dots, b[M - (N - 1)] = \overline{c[1]}, \overline{c[2]}, \dots, \overline{c[N - 1]} \quad (\text{negative-index values, at the back})$$

(the negative-index values equal the same $\overline{c[p]}$ as the positive ones, since $c[-p] = c[p]$ follows from c depending only on n^2). Everything strictly between position $N - 1$ and position $M - (N - 1)$ is zero.

3.7 Full algorithm

1. $M \leftarrow \text{next_power_of_two}(2N - 1)$.
2. Build a, b as above (both length M).
3. $A \leftarrow \text{FFT}(a)$, $B \leftarrow \text{FFT}(b)$ (both power-of-two-length FFTs – your existing radix-2 engine handles these).
4. $Y \leftarrow A \cdot B$ (elementwise).
5. $y \leftarrow \text{IFFT}(Y)$; keep only $y[0], \dots, y[N - 1]$ (the rest is convolution "tail" past what the original problem needed).
6. $X[k] \leftarrow c[k] \cdot y[k]$ for $k = 0, \dots, N - 1$.

3.8 Worked numeric example, $N = 3$ (not a power of two)

Let $x = [1, 1, 0]$.

Reference (direct DFT).

$$X[k] = \sum_{n=0}^2 x[n] e^{-j2\pi kn/3} = 1 + e^{-j2\pi k/3}$$

$$X[0] = 2, \quad X[1] = 0.5 - 0.866j, \quad X[2] = 0.5 + 0.866j$$

Via Bluestein. Chirp: $c[n] = e^{-j\pi n^2/3}$ gives $c = [1, 0.5-0.866j, -0.5+0.866j]$.

$a[n] = x[n]c[n] = [1, 0.5-0.866j, 0]$, zero-padded to $M = 8$ (next power of two $\geq 2(3) - 1 = 5$).

b : $b[0] = 1$; $b[1] = \overline{c[1]} = 0.5+0.866j$; $b[2] = \overline{c[2]} = -0.5-0.866j$; then the negative-index mirror at the tail: $b[7] = \overline{c[1]} = 0.5+0.866j$, $b[6] = \overline{c[2]} = -0.5-0.866j$; positions 3, 4, 5 are zero.

Carrying this through $A = \text{FFT}(a)$, $B = \text{FFT}(b)$, $y = \text{IFFT}(A \cdot B)$, and $X[k] = c[k] \cdot y[k]$ for $k = 0, 1, 2$ (numerically verified) reproduces:

$$X[0] = 2.000, \quad X[1] = 0.500 - 0.866j, \quad X[2] = 0.500 + 0.866j$$

matching the direct DFT to machine precision. This confirms the entire pipeline – chirp construction, padding, convolution via FFT, and the final chirp multiplication – end to end.

3.9 Why the inverse needs no separate implementation

Because Bluestein’s forward transform is built entirely from calls to an existing power-of-two FFT (Section 3.7, steps 3 and 5), its inverse can reuse the identical conjugate-transform-conjugate identity used for any correct forward DFT:

$$x[n] = \frac{1}{N} \overline{\text{DFT}\{\overline{X[k]}\}[n]}$$

This identity depends only on the DFT’s sign convention, not on how the forward transform is internally computed – so it works whether the ”DFT” inside it is the direct definition, a radix-2 FFT, or a full Bluestein transform. No second Bluestein-inverse algorithm needs to be written.

4 Summary Table

Step	Key fact
DFT-of-convolution swap	Valid because $e^{-j2\pi km/N}$ is periodic in m with period N , so m and $(p+n) \bmod N$ are interchangeable inside the exponential.
Convolution Theorem result	$Y[k] = X[k]H[k]$ – circular convolution in time becomes pointwise multiplication in frequency.
Linear vs. circular	Zero-pad to $\geq N_x + N_h - 1$ so wraparound only ever touches zeros.
Bluestein’s identity	$kn = \frac{1}{2}(n^2 + k^2 - (k-n)^2)$ turns a non-factoring product into three independently-factoring squares.
Chirp	$c[n] = e^{-j\pi n^2/N}$; modulates x into a , and its conjugate forms the convolution kernel b .
Padding length	$M = \text{next_power_of_two}(2N - 1)$, required because $(k - n)$ spans $2N - 1$ distinct values.
Inverse	Same conjugate-transform-conjugate trick as any DFT – no new algorithm needed.